HDOC Day-56

Finding the First Negative Integer in Each Sliding Window Using Functions and Pointers in C

Aim

To write a C program using a variable-length integer array, pointers, a user-defined function, and nested loops to find the first negative integer in every contiguous window of size k. Display 0 for a window without a negative integer and -1 for an invalid k.

Algorithm

1. Read and validate the positive array size.
2. Declare a variable-length integer array and read its elements.
3. Read the window size k and call first_negative(arr, size, k), passing the array address.
4. If k is not between 1 and size inclusive, print -1 and return.
5. For each window starting at i from 0 to size-k, initialize first to 0.
6. Use an inner loop from i to i+k-1 to find the first negative value; store it and break.
7. Print the value for each window in left-to-right order, separated by spaces.
8. Stop the program.

C Program

```c
#include <stdio.h>

void first_negative(int *arr, int size, int k);

int main()
{
    int size, k, i;

    printf("Enter the size of array: ");
    scanf("%d", &size);

    if (size <= 0)
    {
        printf("Invalid array size!\n");
        return 0;
    }

    int arr[size];

    printf("Enter %d array elements: ", size);
    for (i = 0; i < size; i++)
        scanf("%d", &arr[i]);

    printf("Enter the value of k: ");
    scanf("%d", &k);

    first_negative(arr, size, k);
    return 0;
}

/* Function to find first negative in each window */
void first_negative(int *arr, int size, int k)
{
    int i, j, first;

    if (k <= 0 || k > size)
    {
        printf("First Negative Elements: -1\n");
        return;
    }

    printf("\nFirst Negative Elements: ");
    for (i = 0; i <= size - k; i++)
    {
        first = 0;
        for (j = i; j < i + k; j++)
        {
            if (*(arr + j) < 0)
            {
                first = *(arr + j);
                break;
            }
        }

        printf("%d", first);
        if (i < size - k)
            printf(" ");
    }
    printf("\n");
}
```

Explanation

The array is passed to first_negative() through its address. The expression *(arr + j) accesses arr[j]. For each sliding window, the inner loop scans from left to right and stops at the first negative value. If none is found, the result remains 0.

Example: arr = 12 -1 -7 8 -15 30 16 28, size = 8, k = 3

Window	Elements	First negative
2	-1 -7 8	-1
4	8 -15 30	-15
6	30 16 28	0

Expected output: First Negative Elements: -1 -1 -7 -15 -15 0

Test Case Table

Test	Input array	k	Expected output
1	12 -1 -7 8 -15 30 16 28	3	-1 -1 -7 -15 -15 0
2	1 2 3 4 5	3	0 0 0
3	-5 -4 -3 -2 -1	3	-5 -4 -3
4	1 -2 3 -4 5	2	-2 -2 -4 -4
5	-1 2 -3 4 -5	1	-1 0 -3 0 -5
6	10 20 -30 40	4	-30
7	5 6 7 8	2	0 0 0
8	1 2 3 4 5	0	-1
9	1 2 3 4 5	6	-1
10	size = 0	-	Invalid array size

Sample Outputs

Test Case 1: Mixed values

```
Enter the size of array: 8
Enter 8 array elements: 12 -1 -7 8 -15 30 16 28
Enter the value of k: 3

First Negative Elements: -1 -1 -7 -15 -15 0
```

Test Case 2: No negative values

```
Enter the size of array: 5
Enter 5 array elements: 1 2 3 4 5
Enter the value of k: 3

First Negative Elements: 0 0 0
```

Test Case 3: All negative values

```
Enter the size of array: 5
Enter 5 array elements: -5 -4 -3 -2 -1
Enter the value of k: 3

First Negative Elements: -5 -4 -3
```

Test Case 4: Invalid k

```
Enter the size of array: 5
Enter 5 array elements: 1 2 3 4 5
Enter the value of k: 6
First Negative Elements: -1
```

Test Case 5: Invalid array size

```
Enter the size of array: 0
Invalid array size!
```

Result

The C program was designed using a variable-length array, pointers, a user-defined function, nested loops, and conditional statements. It displays the first negative integer in each sliding window in order, 0 for windows with no negative integer, and -1 for an invalid k.